EXERCICE 14 · CHANGEMENTS ENTRE LES ÉTAPES

Le panier avec Redux Toolkit

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Point de départ** · template (11 fichiers, en entier)
2. **Étape 1 → 2** · correction (6 fichiers)

Départ Point de départ : template

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/styles.css (247 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @reduxjs/toolkit ^2.11.2, react-redux ^9.2.0

DÉPART **src/main.jsx** 4 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

DÉPART **src/App.jsx** 9 lignes

```jsx
import "./App.css";
import Home from "./pages/Home";

// TODO (exercice) : entourer <Home /> avec le <Provider store={store}> de react-redux
function App() {
  return <Home />;
}

export default App;
```

DÉPART **src/App.css** 16 lignes

```css
/* Import de la police Google Fonts "Inter" avec différentes épaisseurs */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

/* Reset global pour tous les éléments */
* {
  box-sizing: border-box; /* Inclut padding et border dans la largeur/hauteur totale */
  margin: 0; /* Supprime toutes les marges par défaut */
  padding: 0; /* Supprime tous les paddings par défaut */
}

body {
  font-family: "Inter", sans-serif; /* Applique la police Inter à tout le texte */
  -webkit-font-smoothing: antialiased; /* Rend le texte plus lisse sur les écrans */
  background-color: #111827; /* Couleur de fond gris très foncé */
  color: #f9fafb; /* Couleur du texte blanc cassé */
}
```

DÉPART **src/pages/Home.jsx** 18 lignes

```jsx
import "../styles.css";
import Header from "../components/Header";
import ProductList from "../components/ProductList";
import Cart from "../components/Cart";

function Home() {
  return (
    <main className="page">
      <Header />
      <div className="layout">
        <ProductList />
        <Cart />
      </div>
    </main>
  );
}

export default Home;
```

DÉPART **src/components/Header.jsx** 15 lignes

```jsx
function Header() {
  // TODO (exercice) : lire le nombre d'articles du panier dans le store (useSelector)
  const count = 0;

  return (
    <header className="header">
      <h1 className="header-title">Apple Store</h1>
      <div className="cart-badge">
        🛒 Panier <span className="cart-count">{count}</span>
      </div>
    </header>
  );
}

export default Header;
```

DÉPART **src/components/ProductList.jsx** 17 lignes

```jsx
import products from "../data/products";
import ProductCard from "./ProductCard";

function ProductList() {
  return (
    <section>
      <h2 className="section-title">Nos produits</h2>
      <div className="products">
        {products.map((product) => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </section>
  );
}

export default ProductList;
```

DÉPART **src/components/Cart.jsx** 46 lignes

```jsx
import { formatPrice } from "../utils";

function Cart() {
  // TODO (exercice) : lire les lignes et le total du panier dans le store (useSelector)
  const items = [];
  const total = 0;
```

```jsx

  // TODO (exercice) : brancher les boutons −, +, × et « Vider le panier » (useDispatch)

  return (
    <aside className="card cart">
      <h2 className="section-title">Mon panier</h2>

      {items.length === 0 ? (
        <p className="empty">Votre panier est vide.</p>
      ) : (
        <>
          <ul className="cart-lines">
            {items.map((item) => (
              <li key={item.id} className="cart-line">
                <span className="cart-name">
                  {item.icon} {item.name}
                </span>
                <div className="qty">
                  <button>−</button>
                  <span>{item.quantity}</span>
                  <button>+</button>
                </div>
                <span className="cart-price">{formatPrice(item.price * item.quantity)}</span>
                <button className="remove">×</button>
              </li>
            ))}
          </ul>

          <div className="row total">
            <span>Total</span>
            <span className="price">{formatPrice(total)}</span>
          </div>
          <button className="btn btn-secondary">Vider le panier</button>
        </>
      )}
    </aside>
  );
}

export default Cart;
```

DÉPART **src/components/ProductCard.jsx** 28 lignes

```jsx
import { formatPrice } from "../utils";

function ProductCard({ product }) {
  // TODO (exercice) : au clic, dispatcher l'action addToCart(product) (useDispatch)
  const handleAdd = () => {
    console.log("Ajouter au panier :", product.name);
  };

  return (
    <div className="card">
      <div className="product-icon">{product.icon}</div>
      <h3 className="product-name">{product.name}</h3>
      <div className="color">
        <span className="circle" style={{ backgroundColor: product.colorCode }}></span>
        <span>{product.color}</span>
      </div>
      <span className={`badge ${product.available ? "available" : "unavailable"}`}>
        {product.available ? "Available" : "Not Available"}
      </span>
      <p className="price">{formatPrice(product.price)}</p>
      <button className="btn" disabled={!product.available} onClick={handleAdd}>
        Ajouter au panier
```

```
      </button>
    </div>
  );
}

export default ProductCard;
```

DÉPART `src/data/products.js` — 11 lignes

```
// Les produits de la boutique (données en dur, comme un catalogue)
const products = [
  { id: 1, name: "MacBook Air 13\"", icon: "💻", color: "Minuit", colorCode: "#1f2a44", price: 1199, available: true },
  { id: 2, name: "iPhone 16", icon: "📱", color: "Bleu", colorCode: "#7aa6d6", price: 969, available: true },
  { id: 3, name: "AirPods Pro 2", icon: "🎧", color: "Blanc", colorCode: "#f5f5f7", price: 279, available: true },
  { id: 4, name: "Apple Watch Series 10", icon: "⌚", color: "Argent", colorCode: "#c0c4c8", price: 449, available: true },
  { id: 5, name: "iMac 24\"", icon: "🖥️", color: "Violet", colorCode: "#9b8bd6", price: 1499, available: false },
  { id: 6, name: "AirTag", icon: "📍", color: "Blanc", colorCode: "#f5f5f7", price: 35, available: true },
];

export default products;
```

DÉPART `src/utils.js` — 3 lignes

```
// Formate un prix en euros : 1199 => "1 199 €"
export const formatPrice = (price) =>
  price.toLocaleString("fr-FR", { style: "currency", currency: "EUR", maximumFractionDigits: 0 });
```

DÉPART `index.html` — 13 lignes

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Apple Store</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Étape 1 → 2 template → correction

Redux Toolkit branché : slice du panier, store, Provider, useSelector et useDispatch dans les composants

MODIFIÉ `src/App.jsx` +7 −2

```
@@ -1,9 +1,14 @@
  import "./App.css";
  import Home from "./pages/Home";
+ import { Provider } from "react-redux";
+ import { store } from "./store/store";

- // TODO (exercice) : entourer <Home /> avec le <Provider store={store}> de react-redux
  function App() {
-   return <Home />;
+   return (
+     <Provider store={store}>
+       <Home />
+     </Provider>
+   );
  }

  export default App;
```

MODIFIÉ `src/components/Cart.jsx` +20 −9

```
@@ -1,11 +1,18 @@
+ import { useDispatch, useSelector } from "react-redux";
+ import {
+   clearCart,
+   decrement,
+   increment,
+   removeFromCart,
+   selectCartItems,
+   selectCartTotal,
+ } from "../store/cartSlice";
  import { formatPrice } from "../utils";

  function Cart() {
-   // TODO (exercice) : lire les lignes et le total du panier dans le store (useSelector)
-   const items = [];
-   const total = 0;
-
-   // TODO (exercice) : brancher les boutons −, +, × et « Vider le panier » (useDispatch)
+   const items = useSelector(selectCartItems);
+   const total = useSelector(selectCartTotal);
+   const dispatch = useDispatch();

    return (
      <aside className="card cart">
@@ -22,12 +29,14 @@
                    {item.icon} {item.name}
                  </span>
                  <div className="qty">
-                   <button>−</button>
+                   <button onClick={() => dispatch(decrement(item.id))}>−</button>
                    <span>{item.quantity}</span>
-                   <button>+</button>
+                   <button onClick={() => dispatch(increment(item.id))}>+</button>
                  </div>
                  <span className="cart-price">{formatPrice(item.price * item.quantity)}</span>
```

```
30        -                 <button className="remove">×</button>
          37 +              <button className="remove" onClick={() => dispatch(removeFromCart(item.id))}>
          38 +                ×
          39 +              </button>
31        40              </li>
32        41            ))}
33        42          </ul>
          @@ -36,7 +45,9 @@
36        45            <span>Total</span>
37        46            <span className="price">{formatPrice(total)}</span>
38        47          </div>
39        -           <button className="btn btn-secondary">Vider le panier</button>
          48 +        <button className="btn btn-secondary" onClick={() => dispatch(clearCart())}>
          49 +          Vider le panier
          50 +        </button>
40        51        </>
41        52      )}
42        53    </aside>
```

MODIFIÉ **src/components/Header.jsx** +5 −2

```
          @@ -1,6 +1,9 @@
          1 + import { useSelector } from "react-redux";
          2 + import { selectCartCount } from "../store/cartSlice";
          3 +
1         4   function Header() {
2         -   // TODO (exercice) : lire le nombre d'articles du panier dans le store (useSelector)
3         -   const count = 0;
          5 +   // Le nombre d'articles vient du store : il se met à jour partout en même temps
          6 +   const count = useSelector(selectCartCount);
4         7
5         8     return (
6         9       <header className="header">
```

MODIFIÉ **src/components/ProductCard.jsx** +8 −5

```
          @@ -1,10 +1,9 @@
          1 + import { useDispatch } from "react-redux";
          2 + import { addToCart } from "../store/cartSlice";
1         3   import { formatPrice } from "../utils";
2         4
3         5   function ProductCard({ product }) {
4         -   // TODO (exercice) : au clic, dispatcher l'action addToCart(product) (useDispatch)
5         -   const handleAdd = () => {
6         -     console.log("Ajouter au panier :", product.name);
7         -   };
          6 +   const dispatch = useDispatch();
8         7
9         8     return (
10        9       <div className="card">
          @@ -18,7 +17,11 @@
18        17            {product.available ? "Available" : "Not Available"}
19        18          </span>
20        19          <p className="price">{formatPrice(product.price)}</p>
21        -         <button className="btn" disabled={!product.available} onClick={handleAdd}>
          20 +         <button
          21 +           className="btn"
          22 +           disabled={!product.available}
          23 +           onClick={() => dispatch(addToCart(product))}
          24 +         >
22        25            Ajouter au panier
23        26          </button>
24        27        </div>
```

AJOUTÉ **src/store/cartSlice.js** +62 −0

```
@@ -1,0 +1,62 @@
+ import { createSlice } from "@reduxjs/toolkit";
+
+ // Le panier : une liste de lignes { id, name, price, icon, quantity }
+ const initialState = {
+   items: [],
+ };
+
+ const cartSlice = createSlice({
+   name: "cart",
+   initialState,
+   reducers: {
+     // payload = le produit à ajouter
+     addToCart(state, action) {
+       const product = action.payload;
+       const item = state.items.find((i) => i.id === product.id);
+       if (item) {
+         item.quantity += 1; // déjà dans le panier : on augmente la quantité
+       } else {
+         state.items.push({
+           id: product.id,
+           name: product.name,
+           price: product.price,
+           icon: product.icon,
+           quantity: 1,
+         });
+       }
+     },
+     // payload = l'id du produit
+     increment(state, action) {
+       const item = state.items.find((i) => i.id === action.payload);
+       if (item) item.quantity += 1;
+     },
+     // payload = l'id du produit ; à 0, la ligne disparaît
+     decrement(state, action) {
+       const item = state.items.find((i) => i.id === action.payload);
+       if (!item) return;
+       item.quantity -= 1;
+       if (item.quantity === 0) {
+         state.items = state.items.filter((i) => i.id !== action.payload);
+       }
+     },
+     // payload = l'id du produit
+     removeFromCart(state, action) {
+       state.items = state.items.filter((i) => i.id !== action.payload);
+     },
+     clearCart(state) {
+       state.items = [];
+     },
+   },
+ });
+
+ export const { addToCart, increment, decrement, removeFromCart, clearCart } =
+   cartSlice.actions;
+
+ // Selectors : lire le state du panier depuis n'importe quel composant
+ export const selectCartItems = (state) => state.cart.items;
+ export const selectCartCount = (state) =>
+   state.cart.items.reduce((total, item) => total + item.quantity, 0);
+ export const selectCartTotal = (state) =>
+   state.cart.items.reduce((total, item) => total + item.price * item.quantity, 0);
+
+ export default cartSlice.reducer;
```

AJOUTÉ **src/store/store.js** +9 −0

```
@@ -1,0 +1,9 @@
+ import { configureStore } from "@reduxjs/toolkit";
+ import cartReducer from "./cartSlice";
+
+ export const store = configureStore({
+   reducer: {
+     cart: cartReducer,
+   },
+   devTools: true, // optionnel (par défaut true en dev)
+ });
```